been such a thoroughly nice guy. That seemed to be a common theme among the early agilists. They were without exception generous with their knowledge, generous with their time, and humble about their discoveries and achievements.

I had some great mentors in my early years at ThoughtWorks. I was encouraged to try things I'd never done before—coaching skeptical programmers, reaching out to anxious testers, and engaging with suspicious business folks. This, then, was the context in which BDD was born. It was a response to a triple conundrum: programmers didn't want to write tests; testers didn't want programmers writing tests; and business stakeholders didn't see any value in anything that wasn't production code. I didn't think I was inventing a methodology—I was just trying to teach TDD to a bunch of programmers without scaring off the testers.

I started by changing the words, moving from "tests" to "behavior." This seemed to have a profound effect. Then I wrote some software to explore the idea (I registered the jbehave.org domain on Christmas Eve 2003, much to my wife's dismay), and, the following year, developed the Given-When-Then vocabulary of scenarios with business analyst Chris Matts.

Now it's over a decade later and BDD has gone in some surprising directions. Liz Keogh integrated it with complexity theory; Chris Matts and Olav Maassen evolved it into RealOptions; and, of course, it has spawned a plethora of tools in multiple languages. It even has its own conference!

Given all this, I'm delighted to see the arrival of a comprehensive BDD book. John Smart has taken on a large, fast-moving space and produced a book that is everything I would want it to be. He has managed to capture the essence of what I was driving at with BDD while acknowledging the many others who have been part of the journey. As well as a solid theoretical foundation, *BDD in Action* delivers a thorough treatment of the current state of BDD tools, along with general structural and automation guidance that will most likely outlive many of them.

Other authors and practitioners have covered various aspects of BDD—the RSpec and Cucumber books provide a good background but are necessarily tool-oriented. Gojko Adzic's *Specification by Example* (Manning, 2011) is a masterful treatment of the living documentation aspect of BDD and its intersection with acceptance test-driven development. But this is the first time I've seen all the pieces brought together into a cohesive whole.

The last decade has seen agile methods move into the mainstream, which means the relationship between delivery teams and their stakeholders, and the feedback and communication involved in that, can be the difference between success and failure. BDD provides a means to develop and support that relationship, and *BDD in Action* is a great handbook for technical and non-technical participants alike.

I would like to finish on a cautionary note. BDD is a mechanism for fostering collaboration and discovery through examples. The goal isn't generating acceptance criteria or feature files; it isn't automation; it isn't testing; although BDD contains elements of all of these. I see teams obsessing about the number or density of feature